Course Project - Deliverable 2

CSI 2132 – Database I

Winter 2024

University of Ottawa

Faculty of Engineering

School of Electrical Engineering and Computer Science

Course Instructor: Verena Kantere

Jiayi Ma - 300263220

Zixin Fan - 300296371

Mukhveer Kaur - 300317874

Experiment Date: 2024-03-01 to 2024-04-05

A) DBMS and Programming Languages Used

Database Management System(DBMS)

PostgreSQL: We choose PostgreSQL as the database management system. PostgreSQL has strong consistency, reliability, and support for advanced data types, making it an ideal choice for our web applications e-hotels.

Programming Languages and Development Environment

Spring Tool Suite (STS): The development of our web application is powered by Spring Tool Suite, an integrated development environment (IDE) tailored for developing Spring-based enterprise applications. STS enhances developer productivity by providing robust support for Spring application development, including easy access to Spring projects, advanced code completion, and integrated Spring configuration management. This choice reflects our commitment to leveraging Spring's comprehensive ecosystem for creating complex, high-performance web applications.

Java: At the heart of our web application lies Java, a universally employed, object-oriented programming language known for its portability, security, and robust performance. Java's extensive ecosystem, including Spring Framework, makes it an ideal choice for building scalable web applications. Our familiarity with Java further streamlines the development process, allowing us to efficiently tackle complex application logic and backend services.

Frontend Technologies

Our web application's frontend architecture is built upon HTML, CSS, and JavaScript, providing a solid foundation for creating intuitive and visually compelling user interfaces.

HTML, CSS, and JavaScript: The foundation of our web pages is built using HTML, which structures the content. CSS is employed to apply styles and design elements, ensuring that the application is visually appealing. JavaScript is utilized to create interactive and dynamic user experiences, making the web application more engaging and functional. It enables us to create a responsive and engaging user

experience by manipulating DOM elements, handling events, and implementing complex features such as AJAX calls for asynchronous data fetching and updates.

Spring Framework: Integral to our development stack, the Spring Framework provides a comprehensive programming and configuration model for modern Java-based enterprise applications. Spring's vast array of features, including dependency injection, aspect-oriented programming, and transaction management, allows us to build robust, efficient backend services that seamlessly integrate with our frontend technologies.

AJAX: AJAX allows us to asynchronously exchange data with the server and update parts of a web page without reloading the entire page. This method significantly improves the responsiveness and speed of our application, providing a smoother, more dynamic user experience that dynamically reflects changes and user interactions.

B) Installation Guide for the Application

Prerequisites

- Java Runtime Environment (JRE) 8 or higher
- Access to a web server running Apache Tomcat 9 or higher
- A pre-configured PostgreSQL database

Steps

Step1:

1. Download the application WAR file
2. Ensure that Java, Apache Tomcat, and PostgreSQL are properly installed and configured on your server

Step2: Deploy the web application to Tomcat

1. Local machine
    - Navigate to the '**webapps**' directory of your Apache Tomcat installation.
    - Copy the downloaded WAR file into this directory.
    - Start (or restart) the Tomcat server. Tomcat will automatically deploy the WAR file.
2. Remote server

- Access the Tomcat Web Application Manager from your web browser. This is typically found at '**/localhost.com:63342/tutorial12/manager/eHotel/html**'.
- Use the manager's **Deploy** section to upload and deploy the WAR file.

Step3: Configure the database

1. Find the application.properties within the WAR package.
2. Open the file and update it with the local PostgreSQL database details: jdbc:postgresql://localhost:5432/D2.
3. Run psql -d D2 -f path/to/your/script.sql in a terminal to import the initial data into the database.

Step4: Verify the web application

1. Open a web browser and navigate to the web application URL. This is typically '**http://localhost.com:63342/tutorial12/main/eHotel**', where '**eHotel**' is the name of the WAR file, without the '**.war**' extension.
2. You should see the application's CustomerHome page. Try navigating through various sections to ensure it's functioning correctly.

//Backup Approach

Database Setup:

1. Open terminal or command prompt.
2. Import the provided SQL script to set up the database schema and populate it with initial data:'`psql -d D2 -f path/to/your/script.sql`'

Web Application Setup:

1. Deploy the WAR File to Tomcat:
   Copy the downloaded WAR file into the 'webapps' directory of your Apache Tomcat installation.
2. Start Apache Tomcat:
   Run the startup script located in the 'bin' directory of your Apache Tomcat installation.

Running the Application:

1. Access the Application:
   Open a web browser and navigate to CustomerHomePage.html http://localhost.com:63342/eHotel, where 'eHotel' is the name of the deployed WAR file, without the '.war' extension.
2. Explore the Application:

You should see the e-Hotel application's CustomerHome page. Feel free to explore various sections of the application to ensure it's connected to the database and functioning properly.

C) Database Schema Definition

A list of DDLs (click each DDL for the details)

DDLs	Description
create table hotel_chain	table creation
create table hotel	table creation
alter table hotel	• 'zipcode' to be unique • rename 'catagory' to 'category' • change the type of 'category' to numeric • the range of 'category' from 1 to 5
create table phone_number	table creation
alter table phone_number	'phone_number' to be unique
create table email_address	table creation
alter table email_address	'email_address' to be unique
create table employee	table creation
create table room	table creation
alter table room	• 'room_view' to be varchar(100) • 'room_view' has three possibilities • drop 'problems_or_damages'
create table amenities	table creation
create table room_damage	table creation
create table customer	table creation
alter table customer	change the check constraint of 'registration_year'
create table booking	table creation

create table renting_with_booking	table creation
alter table renting_with_booking	• 'renting_id' to be serial • drop all the constraints about dates
create table renting_without_booking	table creation
alter table renting_without_booking	• 'renting_id' to be serial • drop all the constraints about dates • add check constraints to 'check_in/check_out_month', 'check_in/check_out_day', and 'check_in/check_out_hour' again
create table booking_renting_archive	table creation
trigger 1	• **trigger function:** each hotel only has one manager • **trigger implementation:** before inserting one row into the employee table
trigger 2	• **trigger function:** the start_time of booking is after the current time • **trigger implementation:** before inserting one row or updating values on the booking table
trigger 3	• **trigger function:** the end_time of booking is after the start_time of booking • **trigger implementation:** before inserting one row or updating values on the booking table
trigger 4	• **trigger function:** the check_in_time of renting_without_booking is after the current time of renting_without_booking • **trigger implementation:** before inserting one row or updating values on the renting_without_booking table
trigger 5	• **trigger function:** the check_in_time of renting_without_booking is after the current time of renting_without_booking • **trigger implementation:** before inserting one row or updating values

	on the renting_without_booking table
trigger 6	• **trigger function:** the time of renting_with_booking is the same as the corresponding booking time • **trigger implementation:** before inserting one row or updating values on the renting_with_booking table
trigger 7	• **trigger function:** monitor booking, renting_with_booking, and renting_without_booking. based on the operation of these three tables, the archive table gets the data or removes the data automatically. • **trigger implementation:** after inserting one row into the corresponding table
trigger 8	• **trigger function:** when booking is cancelled, which means 'cancelled' is true, the corresponding row will be removed from the archive. • **trigger implementation:** after updating 'cancelled'
index 1	• This index is created to speed up queries that check the cancellation status of bookings based on the booking ID. It is useful for employees to quickly determine whether a specific booking has been cancelled.
index_2	• This index is created to accelerate queries related to date-based searches, such as finding bookings within a specific date range or checking the availability of rooms based on booking dates.
index_3	• This index is created to speed up queries that need to filter rooms based on their availability status. It is useful for employees to quickly find all available or occupied rooms.
index_4	• This index is created to improve the performance of queries that retrieve hotels based on their location. It is convenient for customers to find all

	hotels in a specific zip code area.
index_5	• This index is created to expedite queries related to pricing, such as finding rooms within a specific price range. It is useful for customers to find rooms that meet their budget expectations.
view 1	• This view calculates the number of available rooms in each city by subtracting the number of booked and rented rooms from the total number of rooms in each hotel. It provides a quick overview of room availability for different cities.
view 2	• This view calculates the total capacity of all the rooms in each hotel. It provides a useful summary of the total number of guests that can be accommodated in each hotel.

```
create table hotel_chain(
	Hotel_Chain_ID numeric(1,0) not null check (Hotel_Chain_ID >= 1 and
Hotel_Chain_ID <= 5),
	Number_Of_Hotels numeric(4,0) check (Number_Of_Hotels >= 8),
	Name varchar(20) not null,
	Street varchar(20),
	Unit numeric(4),
	City varchar(20) not null,
	Province varchar(20) not null,
	Zipcode varchar(6) not null check (Zipcode ~ '^[0-9]{6}$' or Zipcode ~
'^[0-9a-zA-Z]{6}$'),
	primary key (hotel_chain_id)
);
```

```
create table hotel(
	hotel_id numeric(12,0) not null check (hotel_id >= 0),
	hotel_chain_id numeric(1,0) not null check (Hotel_Chain_ID >= 1 and
Hotel_Chain_ID <= 5),
	number_of_rooms numeric(4,0) check (number_of_rooms >= 5),
	street varchar(20),
	unit numeric(4),
	city varchar(20) not null,
	province varchar(20) not null,
	zipcode varchar(6) not null check (zipcode ~ '^[0-9]{6}$' or zipcode ~
'^[0-9a-zA-Z]{6}$'),
	catagory varchar(1) not null check (catagory ~ '^[1-5]{1}$'),
	primary key(hotel_chain_id, hotel_id),
	foreign key(hotel_chain_id) references hotel_chain
);

alter table hotel
add constraint unit_zipcode_unique unique (unit, zipcode);--different hotels in the same
building

alter table hotel
rename column catagory to category;

alter table hotel
alter column category
type numeric(1,0) using category::numeric(1,0);

alter table hotel
add constraint category_range_check check (category between 1 and 5);


create table phone_number(
	phone_id SERIAL,
	hotel_chain_id numeric(1,0) not null check (Hotel_Chain_ID >= 1 and
Hotel_Chain_ID <= 5),
	hotel_id numeric(12,0) check (hotel_id >= 0),
	--format +countryNumber(0-2) phoneNumber(7-11)
	phone_number VARCHAR(14) NOT NULL CHECK (
      phone_number ~ '^\+[1-9]\d{0,2}[ ]?\d{7,11}$'
   ),
	primary key(phone_id),
	foreign key(hotel_chain_id) references hotel_chain,
	--
	foreign key(hotel_chain_id, hotel_id) references hotel
);
```

```
alter table phone_number
add constraint phone_number_unique unique (phone_number);

create table email_address(
	email_id serial,
	hotel_chain_id numeric(1,0) not null check (Hotel_Chain_ID >= 1 and
Hotel_Chain_ID <= 5),
	hotel_id numeric(12,0) check (hotel_id >= 0),
	email_address varchar(20) not null check(
		email_address like '%@%.%'
		AND position('@' in email_address) <= position('.' in email_address) - 3
	AND position('.' in email_address) < length(email_address)
	),
	primary key(email_id),
	foreign key (hotel_chain_id) references hotel_chain,
	foreign key(hotel_chain_id, hotel_id) references hotel
);

alter table email_address
add constraint email_unique unique (email_address);

create table employee(
	employee_id varchar(20) not null,
	hotel_chain_id numeric(1,0) not null check (Hotel_Chain_ID >= 1 and
Hotel_Chain_ID <= 5),
	hotel_id numeric(12,0) not null check (hotel_id >= 0),
	id_type varchar(3) not null check (id_type in ('SSN','SIN')),
	family_name varchar(10) not null,
	given_name varchar(10) not null,
	street varchar(20),
	unit numeric(4),
	city varchar(20) not null,
	zipcode varchar(6) not null check (Zipcode ~ '^[0-9]{6}$' or Zipcode ~
'^[0-9a-zA-Z]{6}$'),
	role varchar(20) not null,
	primary key(employee_id),
	foreign key(hotel_chain_id, hotel_id) references hotel,
	constraint check_id_format check(
		(id_type = 'SSN' and employee_id ~ '^\d{3}-\d{2}-\d{4}$')
		or
		(id_type = 'SIN' and employee_id ~ '^\d{3}-\d{3}-\d{3}$')
		or
		(id_type not in ('SSN','SIN'))
	)
);

create table room (
```

```
    room_id NUMERIC(4, 0) not null check(room_id > 0),
    hotel_chain_id numeric(1,0) not null check (Hotel_Chain_ID >= 1 and Hotel_Chain_ID <=
5),
    hotel_id NUMERIC(12, 0) NOT NULL,
            room_view varchar(20) check (room_view in ('sea view', 'mountain view')
                                                                    or room_view is null),
    room_price NUMERIC(6,2) CHECK (room_price >= 100 AND room_price <= 10000) NOT
NULL,
    capacity NUMERIC CHECK (capacity >= 50 AND capacity <= 1000) NOT NULL,
    can_be_extended BOOLEAN NOT NULL,
    problems_or_damages VARCHAR(100),
            status boolean not null,
            primary key (hotel_chain_id, hotel_id, room_id),
    FOREIGN KEY (hotel_chain_id, hotel_id) REFERENCES hotel
);

alter table room drop constraint room_room_view_check;

alter table room
alter column room_view
type varchar(100) using room_view::varchar(100);

alter table room
add constraint view_check check (room_view in ('sea view', 'mountain view',
'sea_and_mountain view'));

alter table room drop column problems_or_damages;

create table amenities (
    amenity_id SERIAL PRIMARY KEY,
            hotel_chain_id numeric(1,0) not null check (Hotel_Chain_ID >= 1 and
Hotel_Chain_ID <= 5),
    hotel_id NUMERIC(12, 0) NOT NULL,
    room_id NUMERIC(4, 0) NOT NULL,
    amenity_name VARCHAR(50) NOT NULL,
    amenity_description VARCHAR(255),
    FOREIGN KEY (hotel_chain_id, hotel_id, room_id) REFERENCES room
);

create table room_damage (
            room_damage_id SERIAL PRIMARY KEY,
            hotel_chain_id numeric(1,0) not null check (Hotel_Chain_ID >= 1 and
Hotel_Chain_ID <= 5),
    hotel_id NUMERIC(12, 0) NOT NULL,
    room_id NUMERIC(4, 0) NOT NULL,
    room_damage_part VARCHAR(50) NOT NULL,
    room_damage_description VARCHAR(255),
```

```
    FOREIGN KEY (hotel_chain_id, hotel_id, room_id) REFERENCES room
);

create table customer (
    customer_id varchar(20) NOT NULL,
    family_name VARCHAR(10) NOT NULL,
    given_name VARCHAR(10) NOT NULL,
    street VARCHAR(20),
    unit NUMERIC(4),
    city VARCHAR(20),
    zipcode VARCHAR(6) CHECK (zipcode ~ '^[0-9]{6}$' OR zipcode ~ '^[0-9a-zA-Z]{6}$'),
    registration_year NUMERIC(4, 0) NOT NULL CHECK (registration_year >= 2000 AND
registration_year <= EXTRACT(YEAR FROM CURRENT_DATE) + 2),
    registration_month NUMERIC(2, 0) NOT NULL CHECK (registration_month >= 1 AND
registration_month <= 12),
    registration_day NUMERIC(2, 0) NOT NULL CHECK (registration_day >= 1 AND
registration_day <= 31),
    ID_type VARCHAR NOT NULL CHECK (ID_type IN ('SSN', 'SIN', 'driving license')),
    PRIMARY KEY (customer_id),
    CONSTRAINT chk_customers_id_format CHECK (
        (ID_type = 'SSN' AND customer_id::TEXT ~ '^\d{3}-\d{2}-\d{4}$') OR
        (ID_type = 'SIN' AND customer_id::TEXT ~ '^\d{3}-\d{3}-\d{3}$') OR
        (ID_type = 'driving license' AND customer_id::TEXT ~ '^A\d{4}-\d{5}-\d{5}$') OR
        (ID_type NOT IN ('SSN', 'SIN', 'driving license'))
    )
);

alter table customer drop constraint customer_registration_year_check;

alter table customer add constraint customer_registration_year_check check
(registration_year >= 2000 AND registration_year <= EXTRACT(YEAR FROM
CURRENT_DATE));

create table booking (
    booking_id SERIAL,
            hotel_chain_id numeric(1,0) not null check (Hotel_Chain_ID >= 1 and
Hotel_Chain_ID <= 5),
    hotel_id numeric(12,0) not null check (hotel_id >= 0),
    room_id NUMERIC(4, 0) NOT NULL check (room_id > 0),
    customer_id varchar(20) NOT NULL,
            cancelled boolean default false,
    start_year NUMERIC(4, 0) NOT NULL CHECK (start_year >= 2000 AND start_year <=
EXTRACT(YEAR FROM CURRENT_DATE) + 2),
    start_month NUMERIC(2, 0) NOT NULL CHECK (start_month >= 1 AND start_month <=
12),
    start_day NUMERIC(2, 0) NOT NULL CHECK (
        (start_day >= 1 AND start_day <= 31) AND
```

```
        ((start_month = 2 AND start_day <= 28) OR
        (start_month IN (4, 6, 9, 11) AND start_day <= 30) OR
        (start_month IN (1, 3, 5, 7, 8, 10, 12) AND start_day <= 31))
    ),
    start_hour NUMERIC(2, 0) NOT NULL CHECK (start_hour >= 0 AND start_hour <= 23),
    end_year NUMERIC(4, 0) NOT NULL CHECK (end_year >= 2000 AND end_year <=
EXTRACT(YEAR FROM CURRENT_DATE) + 2),
    end_month NUMERIC(2, 0) NOT NULL CHECK (end_month >= 1 AND end_month <=
12),
    end_day NUMERIC(2, 0) NOT NULL CHECK (
        (end_day >= 1 AND end_day <= 31) AND
        ((end_month = 2 AND end_day <= 28) OR
        (end_month IN (4, 6, 9, 11) AND end_day <= 30) OR
        (end_month IN (1, 3, 5, 7, 8, 10, 12) AND end_day <= 31))
    ),
    end_hour NUMERIC(2, 0) NOT NULL CHECK (end_hour >= 0 AND end_hour <= 23),
            primary key (booking_id, hotel_chain_id, hotel_id, room_id, customer_id),
    FOREIGN KEY (hotel_chain_id, hotel_id, room_id) REFERENCES room,
    FOREIGN KEY (customer_id) REFERENCES customer
);


create table renting_with_booking(
            renting_id numeric(9,0) not null check (renting_id >= 0),
            hotel_chain_id numeric(1,0) not null check (Hotel_Chain_ID >= 1 and
Hotel_Chain_ID <= 5),
    hotel_id numeric(12,0) not null check (hotel_id >= 0),
    room_id numeric(4, 0) NOT NULL check (room_id > 0),
    customer_id varchar(20) NOT NULL,
    employee_id varchar(20) NOT NULL,
            booking_id serial not null,
            check_in_year numeric(4) check (
                    check_in_year <= extract(year from current_date)
            ),
            check_in_month numeric(2) check (
                    (check_in_month between 1 and 12)
                    and
                    (check_in_month < extract(month from current_date))
            ),
            check_in_day numeric(2) check (
                    ((check_in_month in (1, 3, 5, 7, 8, 10, 12) and check_in_day between 1 and
31)
                    or
                    (check_in_month in (4, 6, 9, 11) and check_in_day between 1 and 30)
                    or
                    (check_in_month = 2 and check_in_day between 1 and 28))
                    and
                    (check_in_day <= extract(day from current_date))
```

```
	),
	check_in_hour numeric(2) check (
		(check_in_hour between 1 and 24)
		and
		(check_in_hour < extract(hour from current_date))
	),
	check_out_year numeric(4) check (
		check_in_year <= extract(year from current_date)
	),
	check_out_month numeric(2) check (
		(check_in_month between 1 and 12)
		and
		(check_in_month < extract(month from current_date))
	),
	check_out_day numeric(2) check (
		((check_in_month in (1, 3, 5, 7, 8, 10, 12) and check_in_day between 1 and
31)
		or
		(check_in_month in (4, 6, 9, 11) and check_in_day between 1 and 30)
		or
		(check_in_month = 2 and check_in_day between 1 and 28))
		and
		(check_in_day <= extract(day from current_date))
	),
	check_out_hour numeric(2) check (
		(check_in_hour between 1 and 24)
		and
		(check_in_hour < extract(hour from current_date))
	),
	primary key (renting_id),
	foreign key (booking_id, hotel_chain_id, hotel_id, room_id, customer_id)
		references booking(booking_id, hotel_chain_id, hotel_id, room_id,
customer_id),
	foreign key (employee_id) references employee
);

ALTER TABLE renting_with_booking DROP COLUMN renting_id;
ALTER TABLE renting_with_booking ADD COLUMN renting_id SERIAL;

ALTER TABLE renting_with_booking DROP CONSTRAINT renting_with_booking_check;
ALTER TABLE renting_with_booking DROP CONSTRAINT renting_with_booking_check1;
ALTER TABLE renting_with_booking DROP CONSTRAINT
renting_with_booking_check_in_hour_check;
ALTER TABLE renting_with_booking DROP CONSTRAINT
renting_with_booking_check_in_hour_check1;
ALTER TABLE renting_with_booking DROP CONSTRAINT
renting_with_booking_check_in_month_check;
```

```
ALTER TABLE renting_with_booking DROP CONSTRAINT
renting_with_booking_check_in_month_check1;
ALTER TABLE renting_with_booking DROP CONSTRAINT
renting_with_booking_check_in_year_check;
ALTER TABLE renting_with_booking DROP CONSTRAINT
renting_with_booking_check_in_year_check1;
```

```
create table renting_without_booking(
        renting_id numeric(9,0) not null check (renting_id >= 0),
        hotel_chain_id numeric(1,0) not null check (Hotel_Chain_ID >= 1 and
Hotel_Chain_ID <= 5),
   hotel_id numeric(12,0) not null check (hotel_id >= 0),
   room_id numeric(4, 0) NOT NULL check (room_id > 0),
   customer_id varchar(20) NOT NULL,
   employee_id varchar(20) NOT NULL,
        check_in_year numeric(4) check (
                check_in_year <= extract(year from current_date)
        ),
        check_in_month numeric(2) check (
                (check_in_month between 1 and 12)
                and
                (check_in_month < extract(month from current_date))
        ),
        check_in_day numeric(2) check (
                ((check_in_month in (1, 3, 5, 7, 8, 10, 12) and check_in_day between 1 and
31)
                or
                (check_in_month in (4, 6, 9, 11) and check_in_day between 1 and 30)
                or
                (check_in_month = 2 and check_in_day between 1 and 28))
                and
                (check_in_day <= extract(day from current_date))
        ),
        check_in_hour numeric(2) check (
                (check_in_hour between 1 and 24)
                and
                (check_in_hour < extract(hour from current_date))
        ),
        check_out_year numeric(4) check (
                check_in_year <= extract(year from current_date)
        ),
        check_out_month numeric(2) check (
                (check_in_month between 1 and 12)
                and
                (check_in_month < extract(month from current_date))
        ),
```

```
	check_out_day numeric(2) check (
		((check_in_month in (1, 3, 5, 7, 8, 10, 12) and check_in_day between 1 and
31)
		or
		(check_in_month in (4, 6, 9, 11) and check_in_day between 1 and 30)
		or
		(check_in_month = 2 and check_in_day between 1 and 28))
		and
		(check_in_day <= extract(day from current_date))
	),
	check_out_hour numeric(2) check (
		(check_in_hour between 1 and 24)
		and
		(check_in_hour < extract(hour from current_date))
	),
	primary key (renting_id),
	foreign key (hotel_chain_id, hotel_id, room_id) references room,
	foreign key (customer_id) references customer,
	foreign key (employee_id) references employee
);

ALTER TABLE renting_without_booking DROP COLUMN renting_id;
ALTER TABLE renting_without_booking ADD COLUMN renting_id SERIAL;


ALTER TABLE renting_without_booking DROP CONSTRAINT
renting_without_booking_check;
ALTER TABLE renting_without_booking DROP CONSTRAINT
renting_without_booking_check1;
ALTER TABLE renting_without_booking DROP CONSTRAINT
renting_without_booking_check_in_hour_check;
ALTER TABLE renting_without_booking DROP CONSTRAINT
renting_without_booking_check_in_hour_check1;
ALTER TABLE renting_without_booking DROP CONSTRAINT
renting_without_booking_check_in_month_check;
ALTER TABLE renting_without_booking DROP CONSTRAINT
renting_without_booking_check_in_month_check1;
ALTER TABLE renting_without_booking DROP CONSTRAINT
renting_without_booking_check_in_year_check;
ALTER TABLE renting_without_booking DROP CONSTRAINT
renting_without_booking_check_in_year_check1;
ALTER TABLE renting_without_booking DROP CONSTRAINT check_in_year;


ALTER TABLE renting_without_booking
ADD CONSTRAINT check_in_month CHECK (check_in_month >= 1 AND check_in_month
<= 12);
```

```
ALTER TABLE renting_without_booking
ADD CONSTRAINT check_in_day CHECK ((check_in_day >= 1 AND check_in_day <= 31)
AND
	((check_in_month = 2 AND check_in_day <= 28) OR
	(check_in_month IN (4, 6, 9, 11) AND check_in_day <= 30) OR
	(check_in_month IN (1, 3, 5, 7, 8, 10, 12) AND check_in_day <= 31)));

ALTER TABLE renting_without_booking
ADD CONSTRAINT check_in_hour CHECK (check_in_hour >= 0 AND check_in_hour <=
23);

ALTER TABLE renting_without_booking
ADD CONSTRAINT check_out_month CHECK (check_out_month >= 1 AND
check_out_month <= 12);

ALTER TABLE renting_without_booking
ADD CONSTRAINT check_out_day CHECK ((check_out_day >= 1 AND check_out_day <=
31) AND
	((check_out_month = 2 AND check_out_day <= 28) OR
	(check_out_month IN (4, 6, 9, 11) AND check_out_day <= 30) OR
	(check_out_month IN (1, 3, 5, 7, 8, 10, 12) AND check_out_day <= 31)));

ALTER TABLE renting_without_booking
ADD CONSTRAINT check_out_hour CHECK (check_out_hour >= 0 AND check_out_hour
<= 23);


create table booking_renting_archive(
		booking_renting_id serial not null,
		hotel_chain_id numeric(1,0) not null check (Hotel_Chain_ID >= 1 and
Hotel_Chain_ID <= 5),
	hotel_id numeric(12,0) not null check (hotel_id >= 0),
	room_id numeric(4, 0) NOT NULL check (room_id > 0),
		customer_id varchar(20) not null,
		employee_id varchar(20),
		archive_type varchar(20) not null check (archive_type in ('booking', 'booking_renting',
'renting')),
		booking_id integer,
		check_in_year numeric(4),
		check_in_month numeric(2),
		check_in_day numeric(2),
		check_in_hour numeric(2),
		check_out_year numeric(4),
		check_out_month numeric(2),
		check_out_day numeric(2),
		check_out_hour numeric(2),
		primary key (booking_renting_id)
);
```

--t1

```
CREATE OR REPLACE FUNCTION check_unique_manager() RETURNS TRIGGER AS $$
BEGIN
  IF EXISTS (
    SELECT 1 FROM employee
    WHERE hotel_chain_id = NEW.hotel_chain_id AND hotel_id = NEW.hotel_id AND role =
'manager'
  ) THEN
    RAISE EXCEPTION 'This hotel already has one manager';
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_check_unique_manager
BEFORE INSERT OR UPDATE ON employee
FOR EACH ROW
WHEN (NEW.role = 'manager')
EXECUTE FUNCTION check_unique_manager();
```

--t2

```
CREATE OR REPLACE FUNCTION check_booking_start_time() RETURNS TRIGGER AS
$$
BEGIN
    IF TO_TIMESTAMP(NEW.start_year || '-' || NEW.start_month || '-' || NEW.start_day || ' ' ||
NEW.start_hour || ':00', 'YYYY-MM-DD HH24:MI') <= CURRENT_TIMESTAMP THEN
        RAISE EXCEPTION 'The start time must be after the current time';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_check_booking_start_before_insert_or_update
BEFORE INSERT OR UPDATE ON booking
FOR EACH ROW EXECUTE FUNCTION check_booking_start_time();
```

--t3

```
CREATE OR REPLACE FUNCTION check_booking_end_after_start() RETURNS TRIGGER
AS $$
BEGIN
    IF TO_TIMESTAMP(NEW.end_year || '-' || NEW.end_month || '-' || NEW.end_day || ' ' ||
NEW.end_hour || ':00', 'YYYY-MM-DD HH24:MI') <= TO_TIMESTAMP(NEW.start_year || '-' ||
```

```
NEW.start_month || '-' || NEW.start_day || ' ' || NEW.start_hour || ':00', 'YYYY-MM-DD
HH24:MI') THEN
        RAISE EXCEPTION 'The end time must be after the start time';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_check_booking_end_after_start_before_insert_or_update
BEFORE INSERT OR UPDATE ON booking
FOR EACH ROW EXECUTE FUNCTION check_booking_end_after_start();
```

--t4

```
CREATE OR REPLACE FUNCTION check_renting_without_booking_start_time()
RETURNS TRIGGER AS $$
BEGIN
    IF TO_TIMESTAMP(NEW.check_in_year || '-' || NEW.check_in_month || '-' ||
NEW.check_in_day || ' ' || NEW.check_in_hour || ':00', 'YYYY-MM-DD HH24:MI') <=
CURRENT_TIMESTAMP THEN
        RAISE EXCEPTION 'The start time must be after the current time';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER renting_without_booking_start_before_insert_or_update
BEFORE INSERT OR UPDATE ON renting_without_booking
FOR EACH ROW EXECUTE FUNCTION check_renting_without_booking_start_time();
```

--t5

```
CREATE OR REPLACE FUNCTION check_renting_without_booking_end_after_start()
RETURNS TRIGGER AS $$
BEGIN
    IF TO_TIMESTAMP(NEW.check_out_year || '-' || NEW.check_out_month || '-' ||
NEW.check_out_day || ' ' || NEW.check_out_hour || ':00', 'YYYY-MM-DD HH24:MI') <=
TO_TIMESTAMP(NEW.check_in_year || '-' || NEW.check_in_month || '-' ||
NEW.check_in_day || ' ' || NEW.check_in_hour || ':00', 'YYYY-MM-DD HH24:MI') THEN
        RAISE EXCEPTION 'The end time must be after the start time';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER renting_without_booking_end_after_start_before_insert_or_update
BEFORE INSERT OR UPDATE ON renting_without_booking
```

```
FOR EACH ROW EXECUTE FUNCTION check_renting_without_booking_end_after_start();
```

--t6

```
CREATE OR REPLACE FUNCTION check_renting_with_booking_times() RETURNS
TRIGGER AS $$
DECLARE
   v_start_year NUMERIC(4, 0);
   v_start_month NUMERIC(2, 0);
   v_start_day NUMERIC(2, 0);
   v_start_hour NUMERIC(2, 0);
   v_end_year NUMERIC(4, 0);
   v_end_month NUMERIC(2, 0);
   v_end_day NUMERIC(2, 0);
   v_end_hour NUMERIC(2, 0);
BEGIN
   SELECT start_year, start_month, start_day, start_hour, end_year, end_month, end_day,
end_hour
   INTO v_start_year, v_start_month, v_start_day, v_start_hour, v_end_year, v_end_month,
v_end_day, v_end_hour
   FROM booking
   WHERE booking_id = NEW.booking_id;

   IF NOT (NEW.check_in_year = v_start_year AND NEW.check_in_month = v_start_month
AND NEW.check_in_day = v_start_day AND NEW.check_in_hour = v_start_hour AND
         NEW.check_out_year = v_end_year AND NEW.check_out_month = v_end_month
AND NEW.check_out_day = v_end_day AND NEW.check_out_hour = v_end_hour) THEN
      RAISE EXCEPTION 'The start_time and end_time of Renting with booking must be the
same as that of booking';
   END IF;

   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_check_renting_with_booking_times
BEFORE INSERT OR UPDATE ON renting_with_booking
FOR EACH ROW EXECUTE FUNCTION check_renting_with_booking_times();
```

--t7

```
create or replace function transfer_to_archive() returns trigger as $$
begin
         if TG_TABLE_NAME = 'booking' then
                  insert into booking_renting_archive (hotel_chain_id, hotel_id, room_id,
customer_id, employee_id, archive_type, booking_id,
```

```
check_in_year, check_in_month, check_in_day, check_in_hour,

check_out_year, check_out_month, check_out_day, check_out_hour)
                values(new.hotel_chain_id, new.hotel_id, new.room_id, new.customer_id,
null, 'booking', new.booking_id,
                        new.start_year, new.start_month, new.start_day, new.start_hour,
                        new.end_year, new.end_month, new.end_day, new.end_hour);
        elsif TG_TABLE_NAME = 'renting_with_booking' then
                insert into booking_renting_archive (hotel_chain_id, hotel_id, room_id,
customer_id, employee_id, archive_type, booking_id,

check_in_year, check_in_month, check_in_day, check_in_hour,

check_out_year, check_out_month, check_out_day, check_out_hour)
                values(new.hotel_chain_id, new.hotel_id, new.room_id, new.customer_id,
new.employee_id, 'booking_renting', null,
                        new.check_in_year, new.check_in_month, new.check_in_day,
new.check_in_hour,
                        new.check_out_year, new.check_out_month, new.check_out_day,
new.check_out_hour);
        elsif TG_TABLE_NAME = 'renting_without_booking' then
                insert into booking_renting_archive (hotel_chain_id, hotel_id, room_id,
customer_id, employee_id, archive_type, booking_id,

check_in_year, check_in_month, check_in_day, check_in_hour,

check_out_year, check_out_month, check_out_day, check_out_hour)
                values(new.hotel_chain_id, new.hotel_id, new.room_id, new.customer_id,
new.employee_id, 'renting', null,
                        new.check_in_year, new.check_in_month, new.check_in_day,
new.check_in_hour,
                        new.check_out_year, new.check_out_month, new.check_out_day,
new.check_out_hour);
        end if;
        return new;
end;
$$ language plpgsql;

create trigger transfer_booking_to_archive after insert on booking
        for each row execute function transfer_to_archive();
create trigger transfer_renting_with_booking_to_archive after insert on renting_with_booking
        for each row execute function transfer_to_archive();
create trigger transfer_renting_without_booking_to_archive after insert on
renting_without_booking
        for each row execute function transfer_to_archive();
```

--t8

```
CREATE OR REPLACE FUNCTION delete_booking_after_cancellation() RETURNS
TRIGGER AS $$
BEGIN
   IF NEW.cancelled THEN
      DELETE FROM booking_renting_archive WHERE booking_id = NEW.booking_id;
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_delete_booking_after_cancellation
AFTER UPDATE OF cancelled ON booking
FOR EACH ROW EXECUTE FUNCTION delete_booking_after_cancellation();
```

INDEX index_booking_id_cancelled

```
CREATE INDEX index_booking_id_cancelled ON booking (booking_id, cancelled);
```

INDEX index_booking_dates

```
CREATE INDEX index_booking_dates ON booking (start_year, start_month, start_day,
start_hour, end_year, end_month, end_day, end_hour);
```

INDEX index_room_status

```
CREATE INDEX index_room_status ON room (status);
```

INDEX index_hotel_zipcode

```
CREATE INDEX index_hotel_zipcode ON hotel (zipcode);
```

INDEX index_room_price

```
CREATE INDEX index_room_price ON room (room_price);
```

VIEW available_rooms_per_city

```
CREATE VIEW available_rooms_per_city AS
SELECT h.city,
      SUM(h.number_of_rooms) - COALESCE(SUM(b.booked_rooms), 0) -
COALESCE(SUM(rwob.rented_rooms), 0) AS available_rooms
FROM hotel h
LEFT JOIN (
   SELECT hotel_id, hotel_chain_id, COUNT(*) AS booked_rooms
   FROM booking
   WHERE cancelled = false
   GROUP BY hotel_id, hotel_chain_id
) b ON h.hotel_id = b.hotel_id AND h.hotel_chain_id = b.hotel_chain_id
LEFT JOIN (
   SELECT hotel_id, hotel_chain_id, COUNT(*) AS rented_rooms
   FROM renting_with_booking
   GROUP BY hotel_id, hotel_chain_id
) rwb ON h.hotel_id = rwb.hotel_id AND h.hotel_chain_id = rwb.hotel_chain_id
LEFT JOIN (
   SELECT hotel_id, hotel_chain_id, COUNT(*) AS rented_rooms
   FROM renting_without_booking
   GROUP BY hotel_id, hotel_chain_id
) rwob ON h.hotel_id = rwob.hotel_id AND h.hotel_chain_id = rwob.hotel_chain_id
GROUP BY h.city;
```

VIEW total_capacity_per_hotel

```
CREATE VIEW total_capacity_per_hotel AS
SELECT hotel_id, SUM(capacity) AS total_capacity
FROM room
GROUP BY hotel_id;
```